

Django 教學 09

表單與資料驗證

第 15 章

從最小範例到專案實作與驗收

本份學習路線

先完成 [08_DTL進階與元件整理](#)。

- **第 15 章：完整表單生命週期**

每章依序：概念、最小範例、語法、專案對照、實作與驗收。

[全課目錄](#) · [實作手冊對照](#)

第 15 章

完整表單生命週期

以有效與無效 POST 驗證表單，觀察 CSRF、錯誤回顯、上傳及成功轉址。

本章的操作環境與成果

先用 LearnBoard 理解表單與擁有權，再對照 LearnMart；兩個專案分別使用自己的資料庫。

完成成果： 以有效與無效 POST 驗證表單，觀察 CSRF、錯誤回顯、上傳及成功轉址。

完整範例可依步驟操作；標示「節錄／重排」的程式用來閱讀，不當作整檔覆蓋。
進階頁可回查，但所有基本驗收需完成。

15-1 為什麼不能直接相信 `request.POST` ？

瀏覽器是「資料來源」，不是可信任邊界。

使用者可以：

- 移除 HTML 的 `required`、`min`、`max`
- 修改 hidden input
- 不經過你的頁面，直接送 HTTP request
- 把數字欄位送成文字或極端值

因此 View 必須在伺服器端驗證，再決定是否改資料。

15-2 HTML form 的四個核心部分

```
<form action="/search/" method="get">  
  <label for="q">關鍵字</label>  
  <input id="q" name="q" value="Django">  
  <button type="submit">搜尋</button>  
</form>
```

- `action` : 送到哪一個 URL
- `method` : GET 或 POST
- `name` : 送出的 key
- `value` : 送出的 value

15-3 GET：查詢，不改變資料

送出後，瀏覽器產生可分享的網址：

```
/?q=Django&category=book
```

Django 讀取：

```
query = request.GET.get("q", "")  
category = request.GET.get("category", "")
```

`request.GET` 是類似 dictionary 的 `QueryDict`。

15-4 GET 參數要保留在畫面上

目前 LearnMart | 節錄／重排 | `templates/base.html`

```
<form action="{% url 'marketplace:home' %}" method="get">
  <input name="q" value="{{ request.GET.q }}"
        placeholder="搜尋商品">
  <button type="submit">搜尋</button>
</form>
```

重新載入結果頁後，搜尋字串仍顯示在 input，使用者知道自己查了什麼。

15-5 POST：要求伺服器改變狀態

適合 POST 的操作：

- 新增／編輯商品
- 加入或更新購物車
- 結帳建立訂單
- 確認出貨
- 發表評價

POST 不是「比較秘密的 GET」；它表示這個 request 可能改變伺服器狀態。

15-6 Form 與 ModelForm

類型	用途	例子
<code>forms.Form</code>	驗證一般輸入，不一定對應資料表	搜尋範圍、聯絡表單
<code>forms.ModelForm</code>	依 Model 欄位建立表單並可儲存 instance	商品、訂單收件資料

先理解 `Form` 的驗證流程，再把相同流程套到 `ModelForm`。

15-7 教學用最小範例 先從 Form 開始

```
from django import forms

class QuantityForm(forms.Form):
    quantity = forms.IntegerField(min_value=1)
```

IntegerField 同時做兩件事：

1. 把文字輸入轉成 Python `int`
2. 檢查值至少為 1

15-8 Unbound form：還沒有收到資料

```
form = QuantityForm()
```

這個 form：

- 尚未綁定任何 request 資料
- `form.is_bound` 是 `False`
- 適合第一次 GET 顯示空表單
- 不會顯示「必填」錯誤

它不是「驗證成功」，而是「還沒進入驗證」。

15-9 Bound form：已經收到資料

```
form = QuantityForm(request.POST)
```

這個 form：

- `form.is_bound` 是 `True`
- 保留使用者送來的值
- 呼叫 `is_valid()` 後產生 `errors` 或 `cleaned_data`
- `invalid` 時應把同一個 bound form 重新 render

15-10 初學時先寫明確的 GET / POST 分支

```
if request.method == "POST":
    form = QuantityForm(request.POST)
    if form.is_valid():
        quantity = form.cleaned_data["quantity"]
        return redirect("marketplace:cart")
else:
    form = QuantityForm()

return render(request, "marketplace/form.html", {"form": form})
```

這比一開始就背 `request.POST or None` 更容易看出狀態轉換。

15-11 為什麼不先教 `request.POST or None` ?

常見縮寫：

```
form = QuantityForm(request.POST or None)
```

但空的 `QueryDict` 是 falsey；一個空 POST 可能被當成 `None`，讓 form 變成 unbound，預期的必填錯誤反而不出現。

教學原則 先用 `request.method` 明確分支；熟悉生命週期後再判斷何時適合縮寫。

15-12 `is_valid()` 做了什麼？

```
if form.is_valid():  
    quantity = form.cleaned_data["quantity"]
```

驗證成功後：

- 必填、型別、範圍等規則通過
- `cleaned_data` 才可安全讀取
- 值已轉成 Python 型別

驗證失敗時，錯誤放在 `form.errors`，不要自行把錯誤字串散落在 View。

15-13 顯示錯誤，不要丟掉使用者輸入

```
<form method="post">
  {% csrf_token %}
  {{ form.as_p }}
  <button type="submit">送出</button>
</form>
```

invalid POST 後要 render 同一個 bound form，Django 才能同時顯示：

- 原本輸入值
- 欄位錯誤
- 非欄位錯誤

15-14 自訂單一欄位驗證

```
from django.core.exceptions import ValidationError

class QuantityForm(forms.Form):
    quantity = forms.IntegerField(min_value=1)

    def clean_quantity(self):
        value = self.cleaned_data["quantity"]
        if value > 20:
            raise ValidationError("單次最多購買 20 件。")
        return value
```

方法名稱必須是 `clean_<field_name>`，且成功時要 return 清理後的值。

15-15 跨欄位驗證使用 `clean()` (1/2)

```
from django.core.exceptions import ValidationError
```

```
class PriceRangeForm(forms.Form):  
    minimum = forms.IntegerField(min_value=0)  
    maximum = forms.IntegerField(min_value=0)
```

單一欄位先各自驗證；`clean()` 再檢查 `minimum` 與 `maximum` 之間的關係。

若其中一個欄位先失敗，它可能不會出現在 `cleaned_data`，所以下一頁不能直接比較兩個 `.get()` 結果。

15-16 跨欄位驗證使用 `clean()` (2/2)

```
def clean(self):
    data = super().clean()
    minimum = data.get("minimum")
    maximum = data.get("maximum")
    if (
        minimum is not None
        and maximum is not None
        and minimum > maximum
    ):
        raise ValidationError("最低價不可高於最高價。")
    return data
```

這個 method 放在 `PriceRangeForm` 內。先呼叫 `super().clean()`、同時 guard 兩個 `None`，沒有 raise 時最後必須 `return data`。

15-17 LearnBoard 的最小 ModelForm

教學節錄 | `board/forms.py` ; 省略 Bootstrap 外觀設定

```
from django import forms
from .models import Message

class MessageForm(forms.ModelForm):
    class Meta:
        model = Message
        fields = ("content",)
```

使用者只填 content ; author 由登入後的伺服器流程指派。

本章先學驗證與儲存，第 16~17 章才把 request.user 與編輯型 CBV 接上。

15-17A BootstrapFormMixin：集中設定欄位外觀

教學簡化 | `board/forms.py`；沿用前頁的 `forms` 匯入

```
class BootstrapFormMixin:
    def apply_bootstrap(self):
        for field in self.fields.values():
            css_class = ("form-select" if isinstance(field.widget, forms.Select)
                        else "form-control")
            field.widget.attrs.setdefault("class", css_class)
```

- Django 預設生成的表單欄位不會自動帶有 Bootstrap class。
- 逐一查看 widget：下拉選單用 `form-select`，其餘用 `form-control`。
- `setdefault()` 只在尚無 `class` 時設定，保留已指定的樣式。

此例省略 checkbox 分支；現有 LearnBoard 另以 `form-check-input` 處理。

15-17B 把 Mixin 接到 MessageForm

教學用組合範例 | 將前頁最小 MessageForm 改為以下定義

```
class MessageForm(BootstrapFormMixin, forms.ModelForm):  
    class Meta:  
        model = Message  
        fields = ("content",)  
  
    def __init__(self, *args, **kwargs):  
        super().__init__(*args, **kwargs)  
        self.apply_bootstrap()
```

先由 `super().__init__()` 建立 `self.fields`，再補上 CSS class。
Mixin 集中可重用的外觀設定；ModelForm 繼續負責欄位、驗證與儲存。

驗收： 在已載入 Bootstrap 的頁面用 `{{ form.as_p }}` 顯示表單，
以 DevTools 確認留言輸入區帶有 `form-control`。商城表單也採相同做法。

15-18 ModelForm 的 Meta

目前 LearnMart | 節錄／重排 | marketplace/forms.py::ProductForm

```
class ProductForm(BootstrapFormMixin, forms.ModelForm):  
    class Meta:  
        model = Product  
        fields = (  
            "category", "name", "description", "price",  
            "stock", "image", "is_active",  
        )
```

Meta 是內部設定 class；fields 是明確 allowlist，不要讓使用者編輯 seller。

15-19 為什麼 `seller` 不在 `fields` ？

如果表單讓使用者選 `seller`，他們就能把商品掛到別人的帳號。

正確來源是已驗證的伺服器資訊：

```
form.instance.seller = request.user
```

安全原則：

角色、擁有者、價格、訂單狀態等 `server-owned` 欄位，不應直接相信瀏覽器提交值。

15-20 `save()` 與 `save(commit=False)`

```
product = form.save(commit=False)
product.seller = request.user
product.save()
```

- `commit=False`：建立尚未寫入資料庫的 instance
- 補上表單沒有提供、但資料庫必須有的欄位
- 最後明確呼叫 `.save()`

目前 LearnMart 的 `CreateView` 使用 `form.instance.seller = self.request.user`，稍後會對照。

15-21 CSRF：防止跨站 mutation 濫用既有 session

```
<form method="post">  
  {% csrf_token %}  
  ...  
</form>
```

`{% csrf_token %}` 會輸出 hidden input。Django 的 CSRF middleware 驗證 cookie/form token，並依 request 情境套用 Origin／Referer policy，阻擋跨站請求濫用既有 session。

CSRF 不代表 browser「可信」，也不負責檢查價格、角色或欄位格式；那些仍要靠表單、權限與商業規則。

15-22 PRG : 成功 POST 後 redirect

```
POST /seller/products/new/  
→ 302 Location: /products/7/  
→ GET /products/7/
```

Post/Redirect/Get 的好處：

- 重新整理結果頁不會重複新增
- URL 變成資源的標準網址
- message 可跨 redirect 顯示一次

invalid POST 則直接 render，讓錯誤留在同一頁。

15-23 Messages：跨 redirect 的一次性回饋

View 端：

```
messages.success(request, "商品已新增。")
```

目前 LearnMart | 節錄／重排 | [templates/base.html](#)

```
{% for message in messages %}
  <div class="alert alert-{{ message.tags|default:'info' }}">
    {{ message }}
  </div>
{% endfor %}
```

message 通常顯示一次，適合 PRG 後回報成功或失敗。

15-24 圖片上傳：HTML 必須改 encoding

```
<form method="post" enctype="multipart/form-data">
  {% csrf_token %}
  {{ form.as_p }}
</form>
```

預設 form encoding 不會傳送檔案內容。 `multipart/form-data` 讓文字欄位與 binary file 一起送出。

目前 `templates/marketplace/form.html` 已使用這個設定。

15-25 Function View 要同時綁定 POST 與 FILES

教學用最小範例

```
if request.method == "POST":
    form = ProductForm(request.POST, request.FILES)
    if form.is_valid():
        product = form.save(commit=False)
        product.seller = request.user
        product.save()
        return redirect(product)
else:
    form = ProductForm()
```

`request.POST` 是一般欄位；`request.FILES` 是上傳檔案。

15-26 POST flow 先畫出來

```
<form method="post">  
↓  
POST /some/url/  
↓  
CSRF middleware  
↓  
View  
↓  
Form(data=request.POST)  
↓ is_valid()  
cleaned_data  
↓  
save / transaction  
↓  
redirect (PRG)
```

Debug 時一次只找一層。

15-27 先觀察 request.POST

開發時可用 debugger / temporary log ；概念上先知道：

```
print(request.POST)
```

會看到 QueryDict 。

真正程式不要把 password/token 等敏感 POST 全部寫進 log 。

15-28 驗證錯誤不等於 exception

Form validation failure 通常是正常的 user input 狀況：

```
if form.is_valid():  
    ...  
else:  
    return render(..., {"form": form})
```

不要把每個 validation error 都變成 HTTP 500。

15-29 CSRF 失敗時先看哪裡？

Template POST form：

```
<form method="post">
  {% csrf_token %}
  ...
</form>
```

403 CSRF 常見原因：

- 漏 `{% csrf_token %}`
- cookie / origin / HTTPS proxy 設定問題
- 用外部工具 POST 卻沒 token

15-30 Browser DevTools 是表單 debugger

Network 面板看：

- Request Method
- Form Data / Payload
- Status Code
- Location response header (302)
- Set-Cookie

不要只看畫面「好像沒反應」。

15-31 概念檢核

1. Unbound form 與 invalid bound form 有何不同？
2. 為什麼 invalid POST 要 render 原本的 form？
3. `cleaned_data` 何時才能讀？
4. `commit=False` 解決什麼問題？
5. CSRF、表單驗證、物件權限各自保護什麼？

答案見配套實作手冊原第 1 章

15-32 LearnMart 實作

任務：追蹤並擴充商品新增流程，確認 seller 一定來自目前登入者。

驗收重點：

- invalid input 顯示錯誤且保留值
- 商品圖片可上傳與顯示
- seller 不可由 POST 任意指定
- 成功後 redirect 並顯示 message

LearnMart 步驟、前後程式碼與驗收命令；LearnBoard 對應練習

第 15 章實作與離堂檢核

任務： 以有效與無效 POST 驗證表單，觀察 CSRF、錯誤回顯、上傳及成功轉址。

1. 展示操作結果或測試紀錄，指出對應檔案與資料。
2. 解釋一個輸入如何得到結果，以及規則在哪一層檢查。
3. 改變一個條件或製造一次失敗，記錄觀察與修正。

配套練習： [LearnBoard 02 原第 1 章](#)；[LearnMart 02 原第 1 章](#)。手冊保留原章號，對照表列出本課位置。

本份完成與後續

下一份：[10_帳號登入與Session](#)。

- 保留本份操作紀錄，確認使用正確的專案與資料庫。
- 章節與實作對應可由 [全課目錄](#) 回查。